Ajaia Docs

Future Scaling & Infrastructure Design

A simple, interview-ready explanation of how the architecture could evolve, why each technology is considered, and what tradeoffs matter.

Status: Future architecture proposal — not infrastructure already implemented. User ranges are illustrative, not measured capacity guarantees.

1. The big picture

Start with the simplest setup that meets current needs. Monitor performance, reliability, cost and product requirements. Add infrastructure only when a specific problem justifies it.

Stage	Approx. concurrent users	Hosting	Database	Main change
0 — Demo	Up to ~100	Vercel	Neon PostgreSQL	One full-stack app
1 — Production	~100–2,000	Docker + ECS Fargate	RDS PostgreSQL	Security, reliability, AWS control
2 — Growing SaaS	~2,000–50,000	Web, collaboration and worker services	Aurora PostgreSQL	Live editing, queues, Redis
3 — Large scale	~50,000–1M+	ECS/EKS, multiple regions	Sharded databases	Global scale and independent services

2. Stage 0 — Our current design

Next.js frontend and REST APIs run on Vercel. Prisma connects to Neon PostgreSQL. Demo signed-cookie sessions identify users. TipTap JSON stores document content. Saves use version checks, and access rules are centralized.

Why Vercel + Neon?	Reason
Fast delivery	Little infrastructure setup
Low initial cost	Free tiers may be sufficient within their limits
Managed operations	Hosting and database management are largely handled
Simple design	One app, one database, one deployment

Alternatives and tradeoffs

Option	Benefit	Why not now?
Vercel + Neon	Quick setup	Selected for demo
EC2	Full host control	Patching and operational work
ECS Fargate	Managed containers	More AWS configuration
Lambda	Serverless execution	Additional integration work for this app
Kubernetes	Advanced orchestration	Unnecessary for one app

Interview: "I chose Vercel and Neon to ship the core product quickly. I would add AWS infrastructure when production needs justify its operational cost."

3. What could become a bottleneck?

Problem	Cause	Possible response
Database connections	More application instances open more connections	Pool connections; consider RDS Proxy
Autosave write load	Many active editors repeatedly save	Reduce writes or send incremental updates
Simultaneous editing	Multiple users change the same document	Yjs and live synchronization
Large documents	Full JSON is sent on every save	Send smaller updates; review storage model

Example: if 1,000 people continuously generate one save each second, that is about 1,000 save requests per second. This is a hypothetical peak; actual load depends on user behavior and debounce timing.

4. Stage 1 — Production on AWS

Proposed request path: Users → Route 53 / CloudFront → Application Load Balancer → multiple ECS Fargate tasks running Next.js → RDS PostgreSQL. ECR stores images; Secrets Manager stores secrets; CloudWatch collects logs and metrics.

Why Docker?

Docker packages the application and runtime dependencies into a versioned image. It improves reproducibility across development, CI and production, though configuration and data can still differ between environments.

Why ECS Fargate?

Option	Advantages	Tradeoffs
ECS Fargate — proposed	Managed containers, scaling, AWS networking	More setup and potentially higher cost
EC2	Control and potentially better cost at steady load	Manage hosts and patching
Lambda	Scales per request	Execution constraints; not a conventional WebSocket host
EKS	Powerful Kubernetes platform	More operational overhead
Stay on Vercel	Excellent developer experience	Less control of private AWS networking

Choose Fargate for managed container operations and AWS networking control, not because it is universally cheaper.

Why RDS instead of Neon?

Both use PostgreSQL. RDS may be preferable when the company needs its database in its own AWS VPC, Multi-AZ high availability, backups and AWS operational integration. Neon remains a valid production option. Moving between them still requires a tested data migration and cutover.

Migration checklist

Step	Change	Purpose
1	Next.js standalone output	Container-friendly server bundle
2	Multi-stage Dockerfile	Build reproducible runtime image
3	Prisma runtime compatibility	Match container platform
4	Docker Compose	Local app + PostgreSQL
5	Health endpoint	Load balancer health checks
6	One-off migration task	Avoid multiple tasks racing to migrate
7	Terraform or AWS CDK	Repeatable VPC, ALB, ECS, RDS, IAM
8	GitHub Actions + ECR	Automated build and deployment
9	Secrets Manager	Keep secrets outside images
10	CloudWatch	Monitor failures and latency
11	Real authentication	Protect customer accounts
12	Rate limiting	Control abusive or excessive requests
13	Document version history	Support recovery

The existing REST endpoints, permission logic and much of the database model can remain, although production authentication and data migration may require application changes.

5. Scaling ECS and database connections

Horizontal scaling adds more ECS tasks; vertical scaling gives tasks more CPU or memory. An ALB routes requests to healthy tasks. ECS Service Auto Scaling can respond to CPU or request metrics.

Connection example: 2 tasks × 10 connections = 20 connections; 20 tasks × 10 = 200. Adding web tasks can exhaust the database connection budget. Set pool limits and consider RDS Proxy if needed.

6. Stage 2 — Real-time collaboration

When users want live co-editing, or 409 conflicts become common, consider separating the ordinary web API from a WebSocket collaboration service. TipTap can integrate with Yjs for concurrent edits.

Current model	Proposed collaboration model
Full-document PATCH saves	Incremental Yjs updates
HTTP request per save	Persistent WebSocket connection
409 on stale version	Merge concurrent changes
Simple backend	Dedicated sync, presence and persistence logic

Why not Yjs from day one? It requires managing connections, synchronization, reconnection, authorization and persistence. Version checks were simpler and appropriate for the take-home.

Why Redis?

If users connected to different collaboration servers edit the same document, servers need to exchange updates. Redis Pub/Sub is one option; it can also support presence and rate limits. Pub/Sub is not durable edit history, so persistent document storage remains necessary.

Why S3, SQS and workers?

Service	Job	Example
S3	Store larger files and snapshots	Original DOCX, exported PDF
SQS	Queue background tasks	Document conversion request
ECS worker	Process tasks outside web request	Convert file, send email, index text

Synchronous import is simpler for small files. Queues become useful when conversion takes too long or requires retries. Workers must handle duplicate messages and failures safely.

Why Aurora, replicas and RDS Proxy?

Aurora can support a growing PostgreSQL workload. Read replicas can serve suitable read queries while the writer handles updates; RDS Proxy helps manage connections. Replicas may lag, so reads requiring immediate consistency may need the writer. Add these only when metrics justify them.

7. Stage 3 — Large-scale architecture

Possible change	Problem it addresses	Cost/complexity
Shard documents across databases	One writer is saturated	Routing, cross-shard queries, migration
Route documents to collaboration servers	Coordinate live edits	Placement and failover
Dedicated permission service	Complex org/team/folder policies	New distributed dependency
Kafka or Kinesis	Event-driven indexing, notifications, analytics	Event ordering and delivery handling
Multi-region deployment	Global latency and regional resilience	Replication and conflict complexity
Consider EKS	Large service platform with specialized needs	Kubernetes operations

Sharding should follow simpler database tuning and scaling. Multi-region active-active is especially complex because edits and permissions must remain consistent across regions.

8. What should trigger an upgrade?

Measured signal or requirement	Possible response
Private AWS networking or compliance requirements	Evaluate Stage 1 hosting and database
Cold starts or connection exhaustion	Improve hosting and connection management
Frequent 409 conflicts	Evaluate Yjs collaboration
High sustained write load	Optimize saves and persistence
Import timeouts	Add SQS workers
Search becomes slow or important	Add indexes, then search service if needed
Single writer saturated after tuning	Evaluate sharding
High cross-region latency	Evaluate multi-region deployment

Do not decide based on user count alone. Track p95 latency, error rates, DB CPU, connection counts, write volume, queue depth and cost.

9. Quick Questions

Why start with Vercel?

It minimizes setup so we can validate the product quickly.

Why Docker?

It makes the application runtime reproducible and portable.

Why Fargate over EC2?

Less host management, in exchange for cost and platform constraints.

Why not Kubernetes?

The initial number of services does not justify its operational overhead.

Why RDS?

AWS networking, operational control and availability requirements—not because Neon cannot scale.

How do you scale the app?

Add ECS tasks based on metrics while monitoring database capacity.

Why Yjs?

It merges concurrent edits and supports live synchronization.

Why Redis?

It can distribute updates between collaboration servers.

Why SQS?

It decouples slow jobs from user-facing requests.

When would you shard?

Only after a single writer remains a measured bottleneck after simpler fixes.